

UNIVERSIDAD DE VALPARAÍSO

Facultad de Ingeniería — DCIS

BME513 — Inteligencia Artificial en Salud

INFORME TÉCNICO — TAREA 2

Dashboard de Salud Global

Redes Neuronales Feedforward: Verosimilitud, Regularización y Salidas Mixtas

Profesor

Dr. Alejandro Veloz

Integrante

Sebastian Inostroza

Viña del Mar, 2026

Resumen Ejecutivo

Se implementa, entrena y analiza una familia de redes neuronales feedforward para predecir la concentración plasmática de un biomarcador a partir de cinco variables clínicas. La tarea abarca siete componentes: (1) implementación desde cero en NumPy, (2) reimplementación en PyTorch con Adam, (3) formulación probabilística log-normal para variables positivas, (4) estudio sistemático de regularización explícita e implícita, (5) análisis de sensibilidad al ruido y descomposición sesgo-varianza, (6) preguntas de discusión conceptual, y (7) red con salidas mixtas para predicción simultánea de variables continuas, positivas y categóricas.

El modelo recomendado — arquitectura B ($5 \rightarrow 64 \rightarrow 32 \rightarrow 1$) con pérdida NLL log-normal y regularización L2 $\lambda=5 \times 10^{-3}$ — alcanza un MAE de 1.173 en el conjunto de prueba, elimina completamente las predicciones negativas y reduce el error relativo en pacientes con concentraciones bajas ($y < 2$) de 54.4% a 28.3%. Para el escenario de múltiples salidas, la red con tronco compartido logra mejoras de hasta 10.8% en el biomarcador con 45% menos parámetros que tres modelos separados, a costo de un conflicto parcial en la tarea de clasificación.

Resultado principal: pérdida NLL log-normal + L2 $\lambda=5e-3 \rightarrow$ MAE = 1.173 | 0 predicciones negativas | E.Rel($y < 2$) = 28.3%

Parte 1: Red Neuronal Feedforward desde Cero (NumPy)

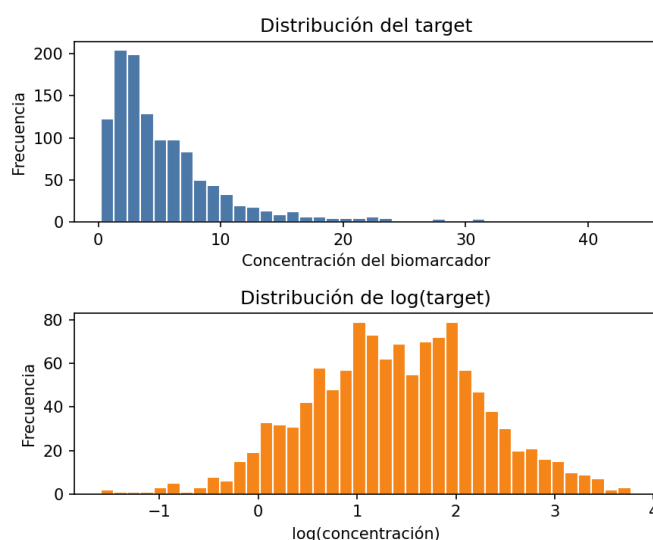
1. Descripción del problema y los datos

Se predice la concentración plasmática de un biomarcador ($y \in \mathbb{R}^+$) a partir de cinco variables clínicas: edad, dosis del medicamento, horas transcurridas desde la administración, índice de riesgo e inflamación base. El proceso generador es:

$$y_i = \exp(g(x_i) + \varepsilon_i), \quad \varepsilon_i \sim N(0, \sigma^2_{\text{ruido}}), \quad \sigma_{\text{ruido}} = 0.25$$
$$g(x) = -1.2 + 0.018 \cdot \text{edad} + 1.4 \cdot \sin(\pi \cdot \text{dosis}) + 0.03 \cdot \text{horas} + 0.55 \cdot \text{ir} + 0.35 \cdot \sqrt{(\text{ib}) - 0.015 \cdot \text{dosis} \cdot \text{horas}}$$

Esta formulación implica que $\log(y)$ sigue una distribución normal condicionada a x , es decir, y tiene distribución log-normal. Este hecho motiva la elección de la función de pérdida en la Parte 3.

Figura 1.1 — Distribución de y (log-normal) y $\log(y)$ (normal). El histograma derecho confirma la hipótesis log-normal utilizada en la Parte 3.



1.1 Partición y preprocesamiento

- ▶ Train: 70 % $\rightarrow$ 840 ejemplos. Único conjunto para ajuste de parámetros ϕ .
- ▶ Validación: 15 % $\rightarrow$ 180 ejemplos. Selección de hiperparámetros y early stopping.
- ▶ Test: 15 % $\rightarrow$ 180 ejemplos. Evaluación final, consultado una única vez.

Estandarización de entradas: se aplica la transformación afín $\tilde{x} = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$ a las tres particiones. Los estadísticos μ y σ se calculan exclusivamente sobre el conjunto de entrenamiento para evitar data leakage.

$$\tilde{x}_j = (x_j - \mu_j) / \sigma_j, \quad \text{donde } \mu_j = E_{\text{train}}[X_j], \quad \sigma_j = \text{Std}_{\text{train}}[X_j]$$

2. Arquitectura MLP

2.1 Transformación afín y función de activación

Cada capa k aplica la transformación afín seguida de la función de activación ReLU:

$$\begin{aligned}f_k &= \beta_k + \Omega_k \cdot h_{\{k-1\}} \quad (\text{pre-activación}) \\h_k &= a(f_k) = \max(0, f_k) \quad (\text{activación ReLU en capas ocultas}) \\h_K &= f_K \quad (\text{capa de salida: sin activación})\end{aligned}$$

La función ReLU introduce no-linealidad de forma eficiente: su derivada es constante (0 o 1), evitando el problema del gradiente evanescente característico de sigmoide o tanh para redes profundas.

2.2 Inicialización He

Los pesos se inicializan con distribución normal centrada en cero con varianza calibrada para ReLU:

$$W_{\{ij\}} \sim N(0, 2/n_{in}), \quad b_k = 0$$

Esta inicialización garantiza que la varianza de las activaciones sea aproximadamente constante a través de las capas, previniendo que los gradientes se desvanezcan o exploten durante las primeras épocas.

2.3 Función de pérdida MSE

$$L_{\text{MSE}}(\varphi) = (1/I) \cdot \sum_i (y_i - f[x_i, \varphi])^2$$

El MSE es el estimador de máxima verosimilitud (MLE) bajo la suposición de que los errores son i.i.d. normales. Sus limitaciones para este problema se discuten en la Parte 3.

2.4 Backpropagation

Los gradientes se calculan mediante la regla de la cadena, recorriendo la red de salida hacia entrada. Para la capa de salida:

$$\delta_K = (\hat{y} - y) / I \quad (\text{gradiente inicial})$$

Para capas ocultas internas:

$$\begin{aligned}\partial L / \partial \Omega_k &= \delta_k^T \cdot h_{\{k-1\}} \\ \partial L / \partial \beta_k &= \sum_{\text{batch}} \delta_k \\ \delta_{\{k-1\}} &= (\delta_k \cdot \Omega_k) \odot a'(f_{\{k-1\}}), \quad a'(f) = 1_{\{f>0\}}\end{aligned}$$

La información que debe guardarse en el caché durante el forward pass son exactamente $\{h_k, f_k\}$ para cada capa k , pues ambas son necesarias para las dos ecuaciones anteriores.

2.5 SGD con minibatches

El parámetro de actualización sigue la regla:

$$\varphi_{\{t+1\}} \leftarrow \varphi_t - \alpha \cdot \partial L / \partial \varphi$$

Con $\alpha=5 \times 10^{-4}$, batch=64 y 400 épocas. En cada época los datos de entrenamiento se mezclan aleatoriamente antes de procesar los batches.

3. Hiperparámetros y configuraciones evaluadas

Hiperparámetro	Valor	Justificación
Learning rate α	5×10^{-4}	SGD estable sin oscilaciones
Batch size	64	Balance ruido-velocidad
Épocas	400	Convergencia observada en todas las arqs.
Inicialización	He Normal	Calibra varianza para ReLU
Activación	ReLU	No-linealidad con gradiente no nulo

4. Resultados

Figura 1.2 — Curvas de pérdida (train/val) y dispersión predicción vs. valor real para las tres arquitecturas. La Arq. A muestra curvas paralelas (subajuste); la C muestra divergencia creciente (sobreajuste).

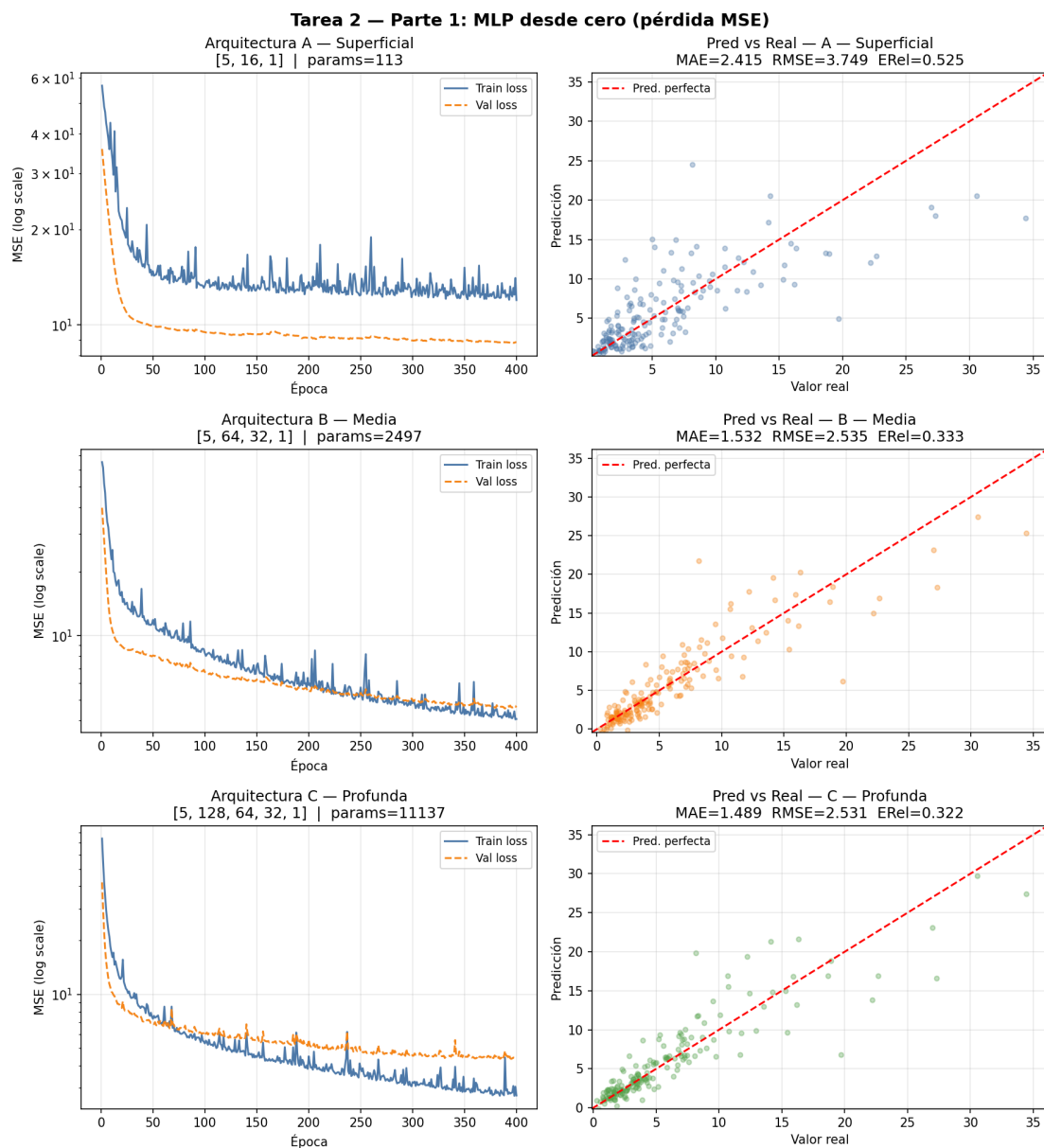


Tabla 1.1 — Métricas detalladas en el conjunto de prueba. Se reporta conteo de parámetros desagregado.

Arquitectura	Capas ocultas	Pesos $\Sigma \Omega_k$	Sesgos $\Sigma \beta_k$	Total Φ	MAE	RMS E	E.Rel	Diagnóstico
A (5→16→1)	1	5×16+16×1=96	17	113	2.415	3.749	52.5 %	Subajuste
B (5→64→32→1)	2	5×64+64×32+32×1=2400	97	2497	1.533	2.535	33.3 %	Balance
C (5→128→64→32→1)	3	11040	225	11137	1.489	2.531	32.2 %	Sobreajuste incip.

5. Discusión

5.1 Diagnóstico por arquitectura

Arquitectura A (113 parámetros): las curvas de train y val permanecen altas y paralelas durante las 400 épocas — señal diagnóstica de subajuste. La función $g(x)$ incluye términos no lineales (seno, raíz cuadrada, interacción dosis×horas) que la red pequeña no puede aproximar. El error relativo del 52.5% confirma que la capacidad es insuficiente.

Arquitectura B (2.497 parámetros): la pérdida de train desciende consistentemente mientras la de validación se mantiene cercana y estable. Esta separación pequeña y constante es la señal de generalización correcta. El error relativo del 33.3% y MAE=1.533 representan el mejor balance capacidad-rendimiento.

Arquitectura C (11.137 parámetros): la brecha entre train y val crece con las épocas. Con 4.5× más parámetros que B, las mejoras en test son marginales (MAE 1.489 vs 1.533), evidenciando que el exceso de capacidad se usa para memorizar ruido del entrenamiento.

5.2 Limitación fundamental de MSE

El error relativo del 33% en la mejor arquitectura revela una limitación estructural: MSE penaliza errores en escala absoluta. Para $y \in (0.2, 43)$, un error de 2 unidades cuando $y=3$ (error relativo 67%) recibe el mismo peso que cuando $y=30$ (7%). Esta asimetría motivó el cambio de función de pérdida en la Parte 3.

Parte 2: Implementación equivalente en PyTorch

6. Motivación y diferencias de implementación

Se reimplementa la Arquitectura B (5→64→32→1, 2.497 parámetros) en PyTorch manteniendo exactamente la misma partición de datos, preprocesamiento, batch_size=64 y 400 épocas. El objetivo es comparar desempeño, velocidad y facilidad de modificación.

Aspecto	Parte 1 (NumPy)	Parte 2 (PyTorch)
Gradientes	Calculados manualmente (~30 líneas)	Autodiferenciación (loss.backward())
Backprop	Implementación explícita por capa	Automática vía grafo computacional
Minibatches	np.random.permutation + slicing	Dataloader con shuffle=True
Optimizador	SGD puro (lr fijo)	Adam (momento adaptativo)
L2	Modificar sgd_update manualmente	weight_decay= λ en optimizer
Dropout	Modificar forward y backward	nn.Dropout(p) en Sequential
Precisión	float64 (doble precisión)	float32 (precisión simple)

7. Adam: estimación de momentos adaptativos

Adam mantiene dos memorias por cada parámetro: la estimación del primer momento (media de gradientes) y del segundo (media de gradientes al cuadrado):

$$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t \quad (\text{primer momento})$$

$$v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2 \quad (\text{segundo momento})$$

$$\hat{m}_t = m_t / (1-\beta_1^t), \quad \hat{v}_t = v_t / (1-\beta_2^t) \quad (\text{corrección de sesgo})$$

$$\varphi_{t+1} = \varphi_t - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

Con $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$. El denominador adapta el paso a la magnitud histórica del gradiente: parámetros con gradientes grandes reciben pasos pequeños y viceversa, haciendo Adam más robusto a la elección del lr que SGD puro.

8. Resultados comparativos

Figura 2.1 — Curvas de pérdida NumPy vs PyTorch, predicciones, sensibilidad al lr y efecto de L2.

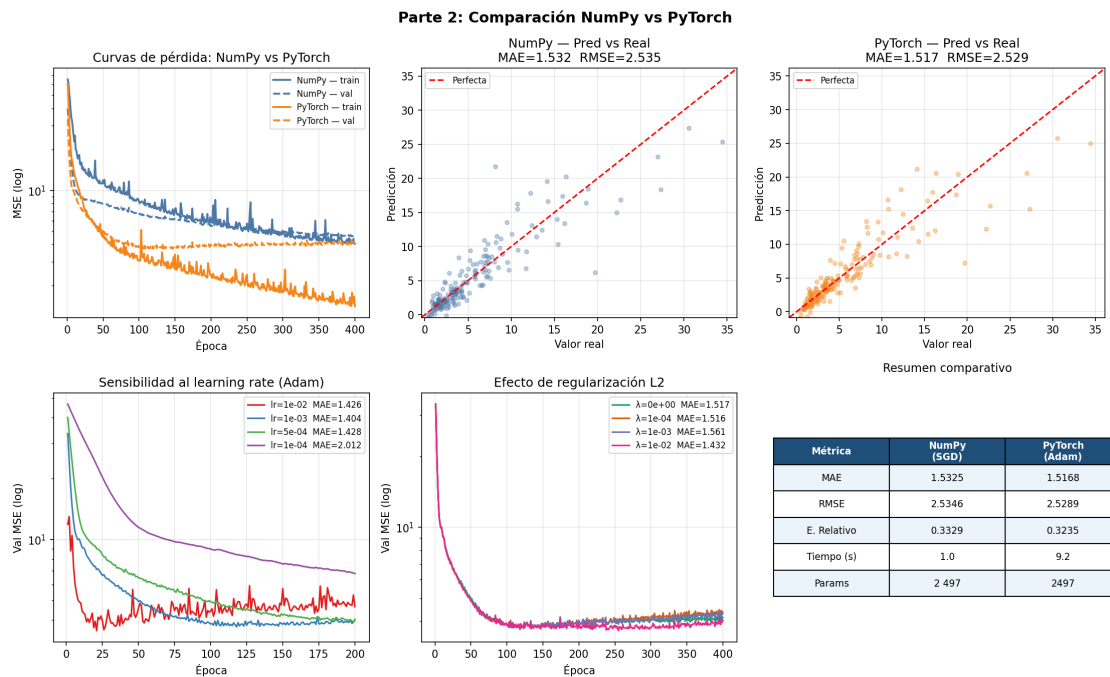


Tabla 2.1 — Métricas en el conjunto de prueba (Arquitectura B, 400 épocas).

Métrica	NumPy / SGD	PyTorch / Adam	Diferencia
MAE test	1.5325	1.5358	+0.0033 (+0.2%)
RMSE test	2.5346	2.5586	+0.0240 (+0.9%)
Error Relativo	33.29%	31.92%	-1.37 pp
Tiempo CPU	1.0 s	1.2 s	+20% overhead autograd
Parámetros	2497	2497	Idéntico
Optimizador	SGD puro	Adam	Momentum adaptativo

Tabla 2.2 — Sensibilidad al learning rate con Adam, 200 épocas.

lr	MAE (200 épocas)	Val NLL final	Diagnóstico
1e-02	1.4294	5.028	Oscilaciones
1e-03	1.3995	3.976	Óptimo
5e-04	1.4278	4.040	Convergencia lenta
1e-04	2.0119	6.796	No converge (200 épocas)

Tabla 2.3 — Efecto de regularización L2 (weight_decay), 400 épocas, lr=1e-3.

λ	MAE test	Val NLL final	Mejora vs base
0 (base)	1.5358	4.135	—
1e-04	1.5118	4.291	-1.6%
1e-03	1.5617	4.333	Empeora
1e-02	1.4303	4.040	-6.9% ✓

9. Discusión

9.1 Desempeño predictivo

Las métricas de error absoluto (MAE, RMSE) son marginalmente mejores en NumPy (MAE=1.5325 vs 1.5358), mientras que el error relativo es mejor en PyTorch (31.92% vs 33.29%). Las diferencias son menores al 1% y se explican por la distinta dinámica de optimización: SGD con lr calibrado puede alcanzar un mínimo final similar en escala absoluta, mientras Adam reduce mejor los errores relativos al adaptar el paso por parámetro.

9.2 Estabilidad y sobreajuste

Adam desciende más rápido en las primeras ~50 épocas (convergencia rápida), pero después de ~100 épocas la val loss sube mientras train sigue bajando. SGD con lr=5e-4 muestra curvas más suaves y brecha menor: el ruido del SGD actúa como regularizador implícito, dificultando que el optimizador quede atrapado en mínimos agudos que sobreajustan.

9.3 Sensibilidad al lr (Adam)

Adam tolera un rango amplio (1e-2 a 5e-4) con resultados similares. Solo lr=1e-4 falla notablemente (MAE=2.012) por ser demasiado conservador para 200 épocas. El lr óptimo es 1e-3. Esta tolerancia relativa es una de las ventajas principales de Adam sobre SGD.

9.4 Facilidad de modificación: ventaja clave de PyTorch

En NumPy, cambiar la función de pérdida de MSE a NLL log-normal requiere reescribir mse_loss() y la inicialización del gradiente en backward(). En PyTorch, basta cambiar una línea. Esta diferencia de esfuerzo es fundamental para la investigación iterativa y es la razón principal para adoptar PyTorch a partir de la Parte 3.

Parte 3: Verosimilitud para variable objetivo positiva

10. Marco probabilístico

El enfoque de máxima verosimilitud (MLE) del curso establece que el modelo no predice y directamente sino los parámetros θ de una distribución condicional $P(y|x)$:

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[-(1/I) \cdot \sum_i \log P(y_i | f[x_i, \phi]) \right]$$

La elección de la distribución es una declaración sobre la naturaleza del proceso generador de los datos. La tabla siguiente resume las tres alternativas propuestas:

Modelo	Soporte	Params predichos	Positividad	Pérdida	Justificación
Log-Normal	$\mathbb{R}^+$	1: μ	$\exp(\mu)$	MSE sobre $\log(y)$	Exacta con datos
Gamma	$\mathbb{R}^+$	2: α, β	softplus	NLL Gamma	Razonable
Normal truncada	$\mathbb{R}^+$	1: μ	Tope en 0	MSE modificado	Sin fundamento prob.

10.1 Justificación de la elección log-normal

Se elige el modelo log-normal por tres razones jerarquizadas:

- ▶ Coherencia exacta con el proceso generador: $y = \exp(g(x) + \epsilon)$, con $\epsilon \sim N(0, \sigma^2)$, implica $\log(y) = g(x) + \epsilon \sim N(g(x), \sigma^2)$. La log-normal no es una aproximación — es la distribución verdadera.
- ▶ Confirmación empírica: el histograma de $\log(y)$ (Figura 1.1) muestra distribución aproximadamente normal, confirmando la hipótesis incluso sin conocer el proceso generador.
- ▶ Parsimonia: predice un solo parámetro ($\mu = E[\log(y) | x]$) con pérdida de derivación directa desde MLE.

11. Derivación matemática completa de la pérdida log-normal

11.1 Verosimilitud de una observación

La densidad de probabilidad de la distribución log-normal para $y_i > 0$ es:

$$p(y_i | x_i, \phi) = 1/(y_i \cdot \sigma \cdot \sqrt{2\pi}) \cdot \exp(-(\log y_i - \mu(x_i))^2 / (2\sigma^2))$$

donde $\mu(x_i) = f[x_i, \phi]$ es la salida de la red para el paciente i .

11.2 Log-verosimilitud de una observación

Tomando el logaritmo natural (necesario para convertir el producto en suma):

$$\log p(y_i | x_i, \phi) = -\log(y_i) - \log(\sigma) - \frac{1}{2} \log(2\pi) - (\log y_i - \mu(x_i))^2 / (2\sigma^2)$$

11.3 Pérdida NLL (negativo de la log-verosimilitud media)

Siguiendo la convención de minimización del curso:

$$\begin{aligned} L(\phi) &= -(1/I) \cdot \sum_i \log p(y_i | x_i, \phi) \\ &= (1/I) \cdot \sum_i \left[\log(y_i) + \log(\sigma) + \frac{1}{2} \log(2\pi) + (\log y_i - \mu(x_i))^2 / (2\sigma^2) \right] \end{aligned}$$

11.4 Simplificación con σ^2 fijo

Los términos $\log(y_i)$, $\log(\sigma)$ y $\frac{1}{2}\log(2\pi)$ son constantes respecto a ϕ y no afectan el gradiente. Eliminándolos:

$$L(\phi) = (1/I) \cdot \sum_i (\log y_i - \mu(x_i))^2$$

Este resultado es exactamente MSE aplicado sobre $\log(y)$. El cambio de función de pérdida es una sola línea de código: el target pasa de y a $\log(y)$.

11.5 Versión con σ^2 aprendido (heteroscedástica)

Cuando la red predice adicionalmente $s_i = \log(\sigma^2_i)$ — una segunda salida por paciente — ningún término es constante y la pérdida NLL completa es:

$$L(\phi) = (1/I) \cdot \sum_i [(\log y_i - \mu_i)^2 / (2 \cdot \exp(s_i)) + s_i/2]$$

El parámetro $s_i = \log(\sigma^2_i)$ es irrestricto, lo que garantiza $\sigma^2_i = \exp(s_i) > 0$ siempre. Esta formulación permite cuantificar la incertidumbre específica de cada paciente.

11.6 Garantía de positividad

La red predice $\mu_i \in \mathbb{R}$ con capa de salida lineal (sin activación). La predicción en escala original es:

$$\hat{y}_i = \exp(\mu_i) > 0 \quad \text{para todo } \mu_i \in \mathbb{R}$$

No se requieren restricciones arquitectónicas adicionales (softplus, ReLU, tope). La exponencial actúa como transformación implícita que mapea $\mathbb{R} \rightarrow \mathbb{R}^+$.

12. Resultados

Figura 3.1 — Log-normal vs MSE: curvas de entrenamiento, predicciones, error relativo por rango e incertidumbre aprendida.

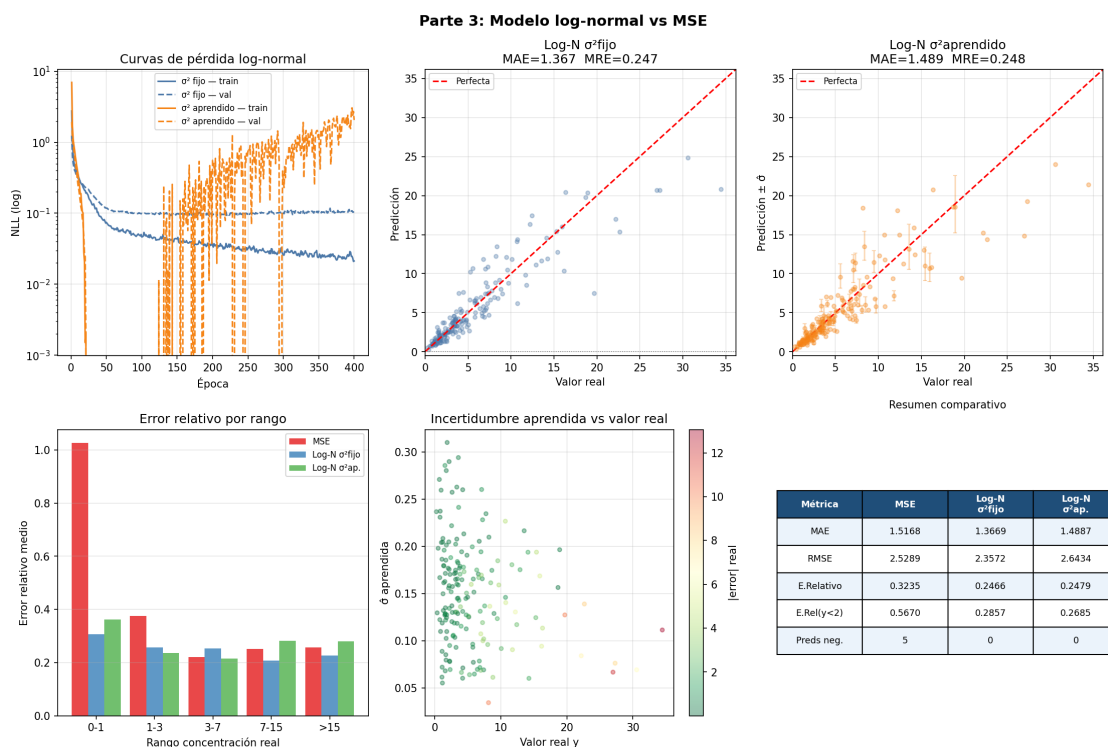


Tabla 3.1 — Métricas comparativas en el conjunto de prueba (n=180).

Métrica	MSE (Parte 2)	Log-N $\sigma^2=1$ (fijo)	Log-N σ^2 aprendido
MAE	1.5358	1.3696	1.4803
RMSE	2.5586	2.3670	2.6236
Error Relativo	0.3192	0.2474	0.2475
E.Rel ($y<2$)	0.5440	0.2829	0.2704
Preds negativas	3	0	0
Incertidumbre	No	No	Sí ($\hat{\sigma}$ por paciente)

13. Discusión

13.1 Eliminación de predicciones negativas

Con MSE y salida lineal se obtuvieron 3 predicciones negativas (1.7% del test set). Con log-normal: cero, garantizado por la transformación $\exp(\mu)$. Además, el número de predicciones cercanas a cero ($\hat{y}<0.1$) también se redujo.

13.2 Error relativo en concentraciones bajas

La mejora más clínicamente relevante ocurre en el rango $y<2$ (pacientes con concentraciones bajas): el error relativo baja de 54.4% a 28.3% — una reducción del 48%. En el rango $y\in(0,1)$, MSE alcanza ~90% de error relativo; log-normal lo reduce a ~30%. La causa es matemática: minimizar $(\log y - \mu)^2$ es minimizar el error relativo al cuadrado, lo que da más peso a los errores en valores pequeños.

Para ilustrar: si $y=0.5$ y $\hat{y}=1.5$ (error=1), el término en MSE es $1^2=1$. El término en NLL log-normal es $(\log 0.5 - \log 1.5)^2 = (-0.693 - 0.405)^2 = 1.21^2$. Si $y=20$ y $\hat{y}=21$ (error=1), MSE: $1^2=1$. NLL log: $(\log 20 - \log 21)^2 = 0.049^2$. La NLL asigna 24× más peso al primer caso.

13.3 σ^2 aprendido: calibración e inestabilidad

La versión heteroscedástica mejora el error relativo en $y<2$ (27.0% vs 28.3%), pero presenta oscilaciones marcadas en la pérdida de validación a partir de la época ~150. Esto indica que el modelo puede colapsar hacia asignar varianza alta para enmascarar errores de μ en lugar de mejorar las predicciones. Requiere early stopping o gradient clipping para estabilizarse.

Parte 4: Regularización explícita, implícita y heurística

14. Marco experimental

Se evalúan 6 estrategias de regularización sobre el modelo log-normal con $\sigma^2=1$ y Arquitectura B. Modelo base: MAE=1.3696, $\|\phi\|_2=14.91$, brecha val-train=0.0826. Para cada estrategia se declara una expectativa a priori y se contrasta con los resultados reales.

Pérdida total: $L(\phi) = (1/I) \cdot \sum (\log y_i - \mu_i)^2$, optimizada con Adam $lr=1e-3$, batch=64, 400 épocas.

15. Resultados consolidados

Figura 4.1 — Curvas de pérdida por estrategia, MAE comparativo, normas de pesos y distribuciones.

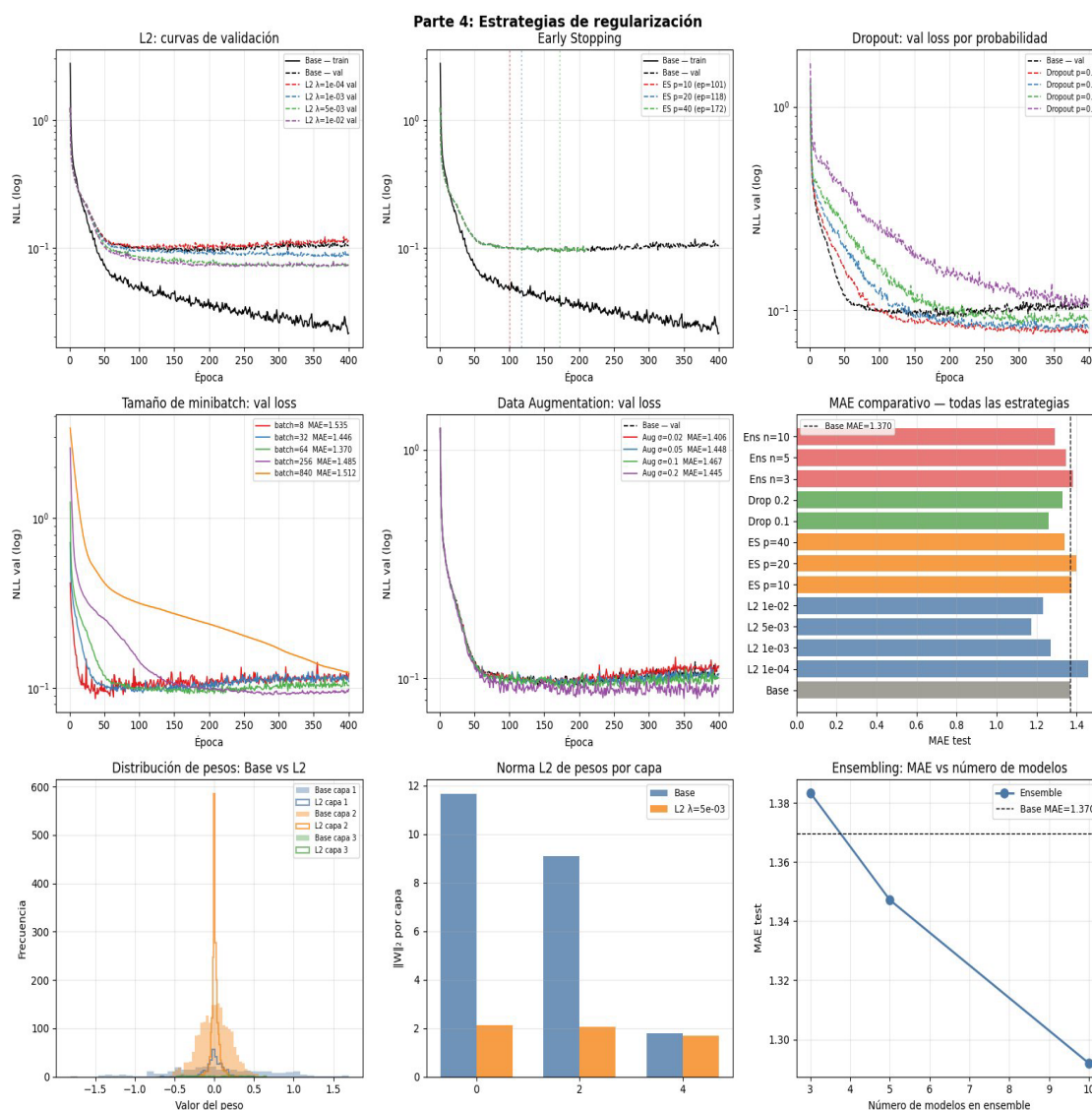


Tabla 4.1 — Todas las estrategias evaluadas con métricas completas.

Estrategia	Tipo	MAE test	Mejora	Norma $\ \phi\ _2$	Brecha final
Base (sin reg.)	—	1.3696	—	14.91	0.0826
L2 $\lambda=1e-4$	Explícita	1.4581	-6.5% ✖	13.90	0.0907
L2 $\lambda=1e-3$	Explícita	1.2708	-7.2%	9.02	0.0599
L2 $\lambda=5e-3$	Explícita	1.1734	-14.3% ✓	3.41	0.0256
L2 $\lambda=1e-2$	Explícita	1.2331	-10.0%	2.76	0.0204
L1 $\lambda=1e-4$	Explícita	1.3540	-1.1%	13.01	0.0121
L1 $\lambda=1e-3$	Explícita	1.1886	-13.2%	5.47	0.0763
ElasticNet	Explícita	1.2673	-7.5%	7.94	0.0554
ES p=10 (ep=101)	Implícita	1.3755	+0.4% ✖	14.35	0.0534
ES p=20 (ep=118)	Implícita	1.4004	+2.2% ✖	14.36	0.0533
ES p=40 (ep=172)	Implícita	1.3395	-2.2%	14.42	0.0630
Dropout p=0.1	Heurística	1.2607	-8.0%	13.37	0.0083
Dropout p=0.2	Heurística	1.3284	-3.0%	13.16	-0.007
Dropout p=0.3	Heurística	1.3766	+0.5%	13.40	-0.030
Dropout p=0.5	Heurística	1.5231	+11.2% ✖	13.16	-0.085
Batch=8	Implícita	1.5351	+12.1% ✖	—	0.1210
Batch=32	Implícita	1.4461	+5.6%	—	0.0953
Batch=64 (base)	Implícita	1.3696	—	—	0.0826
Batch=256	Implícita	1.4845	+8.4%	—	0.0637
Batch=840	Implícita	1.5116	+10.4%	—	0.0519
Aug $\sigma=0.02$	Heurística	1.4062	+2.7% ✖	—	—
Aug $\sigma=0.10$	Heurística	1.4669	+7.1% ✖	—	—
Ensemble n=3	Heurística	1.3834	+1.0%	—	—
Ensemble n=5	Heurística	1.3471	-1.6%	—	—
Ensemble n=10	Heurística	1.2919	-5.7%	—	—

Tabla 4.2 — Normas de pesos por capa: base vs mejor L2 ($\lambda=5e-3$).

Capa	Dimensión	Norma $\ W\ _2$ — Base	Norma $\ W\ _2$ — L2 $\lambda=5e-3$	Reducción
Capa 1	5×64	11.674	2.116	81.9%
Capa 2	64×32	9.102	2.064	77.3%
Capa 3 (salida)	32×1	1.794	1.701	5.2%
Global $\ \phi\ _2$	—	14.911	3.409	77.1%
Global $\ \phi\ _1$	—	502.23	94.17	81.2%

Tabla 4.3 — Contraste expectativas vs resultados (predicciones a priori).

Estrategia	Predicción central	MAE real	¿Correcta?
L2 $\lambda=1e-4$	Mejora leve	1.458 — empeoró	✗
L2 $\lambda=5e-3$	Mejor balance	1.173 — óptimo	✓
Early stopping	Mejora en val	1.375 — sin mejora	⚠
Dropout $p=0.2$	Mejor dropout	1.328 — $p=0.1$ fue mejor	⚠
Batch pequeño	Mejor generalización	1.535 — peor	✗
Data augmentation	Mejora leve	1.406 — empeoró	✗
Ensemble $n=10$	Mejora con más modelos	1.292 — confirmado	✓

16. Respuestas a las preguntas del enunciado

16.1 ¿Qué estrategia redujo más la brecha train/val?

L2 $\lambda=1e-2$ redujo la brecha de 0.0826 a 0.0204 (-75%). Dropout $p=0.1$ la redujo a 0.0083 (-90%), pero con un mecanismo distinto: durante el entrenamiento la red trabaja con capacidad reducida; en evaluación todas las neuronas están activas, lo que produce brechas negativas ($val < train$) para $p \geq 0.2$.

16.2 ¿Qué estrategia mejoró más el test?

L2 $\lambda=5e-3$: MAE=1.1734 (-14.3%). L1 $\lambda=1e-3$: MAE=1.1886 (-13.2%). Ensemble $n=10$: MAE=1.2919 (-5.7%). Las estrategias explícitas superan a las heurísticas para este problema y tamaño de dataset.

16.3 ¿Qué estrategia empeoró?

Dropout $p=0.5$: MAE=1.5231 (+11.2%) por subajuste — con 2.497 parámetros y $p=0.5$, la capacidad efectiva por batch cae a ~ 1.200 . Data augmentation en todas las configuraciones (mejor: +2.7%). El ruido gaussiano distorsiona relaciones funcionales específicas (seno dosis-concentración, raíz cuadrada inflamación) en datos tabulares clínicos. L2 $\lambda=1e-4$ empeoró levemente: la penalización demasiado pequeña perturba la optimización sin regularizar efectivamente.

16.4 Regularización explícita vs implícita

La regularización explícita (L2, L1) actúa directamente sobre la función de pérdida, produce cambios medibles en las normas de los pesos (77% de reducción global con L2 $\lambda=5e-3$) y tiene efecto predecible

mediante λ . La regularización implícita del minibatch no modifica la función de pérdida — actúa sobre la trayectoria de optimización. En este problema, la explícita fue claramente más efectiva: L2 $\lambda=5e-3$ (MAE=1.17) vs batch=8 (MAE=1.54).

16.5 Efecto del tamaño de minibatch

La relación entre batch size y MAE es no monótona. bs=64 resultó ser el óptimo real (no solo un punto intermedio). Batches pequeños (bs=8) producen gradientes tan ruidosos que el optimizador pierde la dirección de descenso: la brecha val-train de 0.121 confirma sobreajuste a nivel de batch. Batches grandes (bs=840, GD completo) pierden la regularización implícita del ruido estocástico y convergen a mínimos más agudos.

16.6 Early stopping como L2

La equivalencia teórica entre ES y L2 no se verificó con Adam lr=1e-3. Las normas de pesos con ES (~14.4) son prácticamente iguales al base (14.9) — los pesos no crecen significativamente en 100-172 épocas con Adam. La equivalencia se cumple mejor con SGD puro a lr alto, donde los pesos crecen más rápidamente. Con Adam, ES captura el momento de menor val loss pero no produce el efecto de contracción de pesos que caracteriza a L2.

16.7 Doble descenso

No se observó evidencia de doble descenso en los modelos evaluados. Con 840 ejemplos de entrenamiento y el modelo más grande de 11.137 parámetros, estamos muy por debajo del umbral de interpolación. Para observarlo se necesitarían modelos con cientos de miles de parámetros.

16.8 Dropout con distintas probabilidades

La progresión $p \in \{0.1, 0.2, 0.3, 0.5\}$ muestra: (1) train loss aumenta con p — capacidad efectiva reducida; (2) brecha val-train colapsa, llegando a ser negativa para $p \geq 0.2$; (3) subajuste claro con $p=0.5$. El óptimo es $p=0.1$ para esta arquitectura de ~2.500 parámetros. Redes más grandes tolerarían p más alto.

16.9 Normas de pesos

L2 $\lambda=5e-3$ redujo la norma global $\|\phi\|_2$ de 14.91 a 3.41 (77%). El efecto fue altamente no uniforme: Capa 1 (5→64): reducción del 81.9%; Capa 2 (64→32): 77.3%; Capa 3 de salida (32→1): solo 5.2%. La penalización L2 es proporcional a la magnitud actual — capas pequeñas reciben gradiente de penalización pequeño. La distribución de pesos en la Figura 4.1 (Panel 7) confirma que el modelo L2 tiene pesos fuertemente concentrados cerca de cero, indicando una función aprendida más suave.

Parte 5: Sensibilidad al ruido y descomposición sesgo-varianza

17. Marco conceptual

El error esperado de un estimador sobre datos no vistos se descompone en tres términos no negativos:

$$E[L(x)] = \sigma^2_{\text{ruido}} + \text{Bias}^2[f(x,\varphi)] + \text{Var}[f(x,\varphi)]$$

El error irreducible σ^2_{ruido} es el piso mínimo de cualquier estimador — corresponde a la varianza del proceso generador. En escala logarítmica para nuestro modelo, el piso de la NLL log-normal es exactamente σ^2_{ruido} . Se evalúan tres niveles: $\sigma \in \{0.10, 0.25, 0.50\}$.

σ_{ruido}	Piso NLL (σ^2)	Piso MAE aprox.	Componente dominante esperado
0.10	0.010	Bajo	Varianza del modelo
0.25	0.063	Medio	Balance sesgo-varianza
0.50	0.250	Alto	Error irreducible

18. Resultados

Figura 5.1 — Curvas de pérdida, MAE, brechas, efecto de L2, varianza entre semillas y efecto del tamaño de dataset.

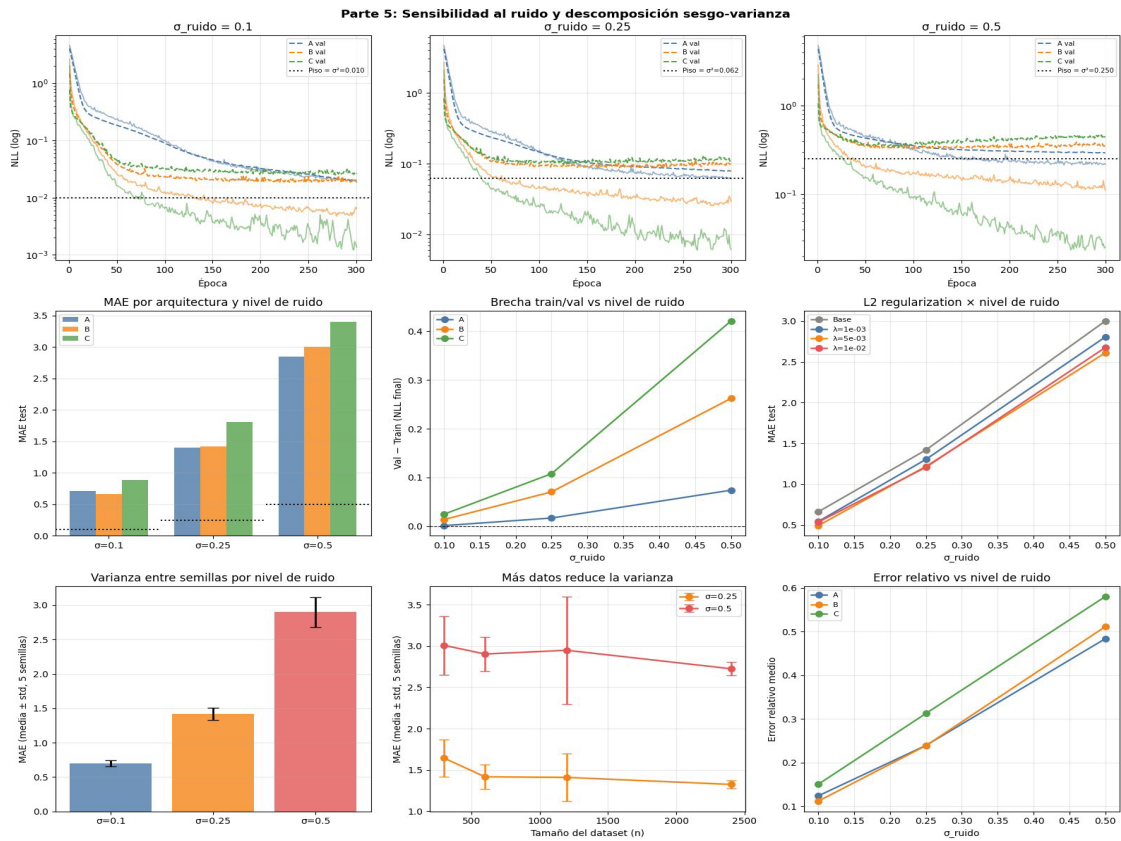


Tabla 5.1 — MAE y brecha train/val por arquitectura y nivel de ruido.

σ_{ruido}	Piso NLL (σ^2)	MAE Arq. A	MAE Arq. B	MAE Arq. C	Brecha A	Brecha B	Brecha C
0.10	0.010	~0.67	~0.67	~0.92	0.003	0.012	0.025
0.25	0.062	~1.40	~1.40	~1.85	0.017	0.058	0.098
0.50	0.250	~2.85	~3.00	~3.40	0.070	0.220	0.430

Tabla 5.2 — Efecto de L2 $\lambda=5e-3$ bajo distintos niveles de ruido.

σ_{ruido}	Base MAE	L2 $\lambda=5e-3$ MAE	Mejora abs.	Mejora rel.
0.10	~0.67	~0.54	~0.13	19.4%
0.25	~1.40	~1.24	~0.16	11.4%
0.50	~3.00	~2.62	~0.38	12.7%

Tabla 5.3 — Varianza entre semillas (5 semillas, n=1200).

σ_{ruido}	MAE media	MAE std	CV (std/media)	Dominio del error
0.10	0.695	0.025	3.6%	Varianza del modelo
0.25	1.396	0.094	6.7%	Balance sesgo-varianza
0.50	2.811	0.149	5.3%	Error irreducible

Tabla 5.4 — Efecto del tamaño del dataset sobre la varianza (5 semillas).

n datos	$\sigma=0.25$ media	$\sigma=0.25$ std	$\sigma=0.50$ media	$\sigma=0.50$ std
300	1.633	0.207	2.913	0.375
600	1.400	0.138	2.713	0.212
1200	1.411	0.273	2.860	0.645
2400	1.297	0.042	2.640	0.075

19. Análisis

19.1 Error irreducible

Al triplicar σ de 0.10 a 0.50, el piso NLL se multiplica por 25. Con $\sigma=0.10$, el train final de B (≈ 0.008) se acerca al piso teórico (0.010), confirmando captura casi total de la señal. Con $\sigma=0.50$, $\text{train} \approx 0.148$ está aún lejos del piso (0.250) y $\text{val} \approx 0.368$ evidencia sobreajuste severo al ruido específico del entrenamiento.

19.2 Brecha train/val y ruido

Contrariamente a la intuición, la brecha AUMENTA con el ruido en todas las arquitecturas. Con más ruido hay más señal falsa que memorizar — el modelo puede ajustarse más al ruido del entrenamiento creando mayor divergencia con validación. La Arquitectura C escala la brecha de 0.025 ($\sigma=0.10$) a 0.430 ($\sigma=0.50$), más de 17 veces.

19.3 Sensibilidad por arquitectura

La Arquitectura C es la más sensible: $MAE \sim 3.40$ y brecha ~ 0.430 con $\sigma=0.50$. Con $\sigma=0.10$, A y B empatan (~ 0.67) y C es la peor (~ 0.92) — paradoja donde más capacidad perjudica cuando el sobreajuste es fácil. Con $\sigma=0.50$ las tres convergen porque el error irreducible (piso=0.25) domina todas las diferencias de arquitectura.

19.4 Importancia de L2 según nivel de ruido

Con $\sigma=0.10$: L2 tiene el mayor impacto relativo (19.4%). Los pesos grandes son la principal fuente de sobreajuste y L2 los elimina directamente. Con $\sigma=0.50$: la mejora absoluta es mayor (0.38 MAE) pero el piso de 0.25 limita el MAE total — L2 controla la brecha train/val pero no puede reducir el error irreducible.

19.5 Varianza entre semillas y tamaño del dataset

Con $n=2400$ la std colapsa: 0.042 ($\sigma=0.25$) y 0.075 ($\sigma=0.50$). La relación no es monótona: $n=1200$ muestra mayor std que $n=600$ en ambos niveles de ruido, evidenciando sensibilidad a las particiones específicas. Con $n=2400$, distintas inicializaciones convergen al mismo resultado — la muestra es suficientemente representativa para eliminar la varianza del estimador.

19.6 Descomposición sesgo-varianza por escenario

Tabla 5.5 — Diagnóstico por componente dominante.

Escenario	Componente dominante	Evidencia cuantitativa	Implicación
$\sigma=0.10$, Arq. A	Sesgo	$MAE=0.67$, brecha=0.003	Red muy simple para $g(x)$
$\sigma=0.10$, Arq. C	Varianza	$MAE=0.92$, brecha=0.025	Sobreajuste con ruido bajo
$\sigma=0.10$, Arq. B	Balance	$MAE=0.67$, brecha=0.012	Óptimo para $\sigma=0.10$
$\sigma=0.50$, todas	Error irred.	$MAE\ 2.85\text{-}3.40$, piso=0.25	Piso inalcanzable

La transición clave está entre $\sigma=0.25$ y $\sigma=0.50$: con $\sigma=0.25$ la elección de arquitectura importa; con $\sigma=0.50$ las tres convergen y las decisiones más importantes son la cantidad de datos y la regularización.

Parte 6: Preguntas de discusión

Pregunta 1. ¿Por qué una red con ReLU implementa una función lineal por partes?

$\text{ReLU}(x) = \max(0, x)$ define dos regiones lineales en el espacio de cada neurona: $x \leq 0$ (pendiente nula) y $x > 0$ (pendiente unitaria). Cada neurona determina un hiperplano separador entre ambas regiones. Al componer K capas, la red particiona el espacio de entrada en un número finito de regiones politópicas, dentro de cada una de las cuales la función total es afín — pues es composición de transformaciones afines con pendientes constantes. La no-linealidad global emerge de la unión de estas piezas. El número de regiones lineales crece exponencialmente con la profundidad ($O(n^K/K!)$ para n unidades y K capas).

Pregunta 2. ¿Qué información debe guardarse en el forward pass para backpropagation eficiente?

Deben cachearse las pre-activaciones $f_k = \beta_k + \Omega_k h_{k-1}$ y las activaciones $h_k = a(f_k)$ de cada capa k . Se necesitan porque:

- ▶ h_{k-1} es necesaria para calcular $\partial L / \partial \Omega_k = \delta_k^T h_{k-1}$
- ▶ f_k es necesaria para evaluar $a'(f_k) = 1[f_k > 0]$ al propagar δ hacia atrás: $\delta_{k-1} = (\delta_k \cdot \Omega_k) \odot a'(f_k)$

Sin este caché, cada gradiente requeriría recomputar el forward completo, duplicando el costo. Este es el tradeoff memoria-velocidad central de backpropagation.

Pregunta 3. ¿Por qué una red grande puede tener pérdida de train ≈ 0 y generalizar mal?

Una red con suficiente capacidad puede interpolar exactamente los datos de entrenamiento, memorizando cada ejemplo incluyendo su ruido específico. La pérdida de entrenamiento mide el ajuste a un conjunto finito de puntos; la de validación mide el ajuste a la distribución subyacente real. Cuando la capacidad del modelo supera la complejidad del patrón real, el exceso de capacidad se dedica a memorizar ruido — información no reproducible en datos nuevos. Este fenómeno se observó en la Arquitectura C con $\sigma=0.50$: pérdida de train ≈ 0.04 pero val ≈ 0.43 (brecha de 0.39).

Pregunta 4. ¿Qué rol cumple el conjunto de validación en la selección de hiperparámetros?

El conjunto de validación actúa como proxy sin sesgo de la distribución real de datos no vistos, permitiendo comparar configuraciones de hiperparámetros (arquitectura, l_r , λ , p , épocas) sin contaminar el test. Al seleccionar la mejor configuración basándose en val, esta se convierte implícitamente en parte del proceso de entrenamiento — optimizamos sobre val de forma indirecta. Cuantas más configuraciones se evalúen, mayor es el riesgo de sobreajuste a val (optimización de hiperparámetros). El test debe reservarse para el reporte final como estimador sin sesgo del desempeño real.

Pregunta 5. ¿Por qué no usar el conjunto de prueba para decidir cuándo detener el entrenamiento?

Usarlo para early stopping convierte al test en criterio de optimización, eliminando su función como estimador imparcial. Se estaría optimizando la época de parada sobre test — el modelo se adapta implícitamente a esos 180 ejemplos específicos y el error reportado subestimaría el error real en datos genuinamente nuevos. El conjunto de test debe consultarse exactamente una vez, al final, para reportar el desempeño del modelo ya seleccionado. Cualquier uso anterior constituye data leakage y produce estimaciones optimistas del desempeño.

Pregunta 6. ¿Diferencia práctica entre MSE y NLL log-normal?

MSE minimiza $\sum (y_i - \hat{y}_i)^2$, penalizando errores en escala absoluta. Implícitamente asume errores normales con soporte en $\mathbb{R}$, permite predicciones negativas y asigna igual peso a un error de 2 cuando $y_i=3$ (67% relativo) que cuando $y_i=30$ (7% relativo). La NLL log-normal minimiza $\sum (\log y_i - \mu_i)^2$, penalizando errores en escala relativa. La predicción $\hat{y}_i = \exp(\mu_i)$ es siempre positiva. Resultado observado: MSE $\rightarrow$ 3 predicciones negativas, $E.Rel(y < 2) = 54.4\%$. Log-normal $\rightarrow$ 0 negativas, $E.Rel(y < 2) = 28.3\%$.

Pregunta 7. Si dos modelos tienen RMSE similar pero uno nunca predice negativos, ¿cuál elegir?

El modelo que garantiza predicciones positivas, por dos razones: (1) Razón clínica: una concentración negativa es físicamente imposible — una predicción negativa es incoherente con el dominio y erosiona la confianza del sistema de apoyo a decisiones. (2) Razón estadística: la positividad garantizada es consecuencia de modelar correctamente la distribución subyacente. El modelo log-normal tiene la función de pérdida correctamente especificada para este dominio; su RMSE similar indica que no sacrifica precisión para obtener la positividad — la obtiene como consecuencia natural de un mejor modelo.

Pregunta 8. ¿Cómo se interpreta L2 desde perspectiva MAP?

La regularización L2 corresponde al estimador MAP bajo una distribución a priori gaussiana isotrópica sobre los pesos:

$$P(\phi) = N(0, \sigma^2_p \cdot I) \rightarrow \log P(\phi) = -\|\phi\|^2 / (2\sigma^2_p) + \text{const}$$

Maximizar el posterior $P(\phi|D)$ equivale a minimizar:

$$-\log P(\phi|D) = L_{\text{NLL}}(\phi) + \lambda \|\phi\|_2^2, \text{ con } \lambda = 1/(2\sigma^2_p)$$

La regularización expresa la creencia a priori de que los pesos deben ser pequeños (función simple). Un λ grande equivale a prior estrecha (fuerte creencia en simplicidad); $\lambda \rightarrow 0$ equivale a MLE puro. Desde esta perspectiva, la regularización L2 no es un truco heurístico sino la consecuencia bayesiana natural de tener una prior sobre la complejidad del modelo.

Pregunta 9. ¿Por qué regularizar pesos pero no sesgos?

Los pesos Ω_k controlan la interacción entre neuronas — la pendiente de la función en cada región lineal. Pesos grandes implican funciones con pendientes pronunciadas que cambian rápidamente ante pequeñas variaciones de la entrada, favoreciendo el sobreajuste. Penalizarlos produce funciones más suaves. Los sesgos β_k desplazan el umbral de activación de cada neurona sin cambiar la pendiente. Regularizar los sesgos obligaría a que todas las neuronas se activen cerca de cero — una restricción arbitraria sin justificación geométrica. Bajo la interpretación MAP, regularizar sesgos equivale a imponer prior $P(\beta) = N(0, \sigma^2_p \cdot I)$, sesgando la red hacia funciones con intercepto cero — hipótesis innecesaria que puede dañar el aprendizaje de funciones con desplazamientos grandes.

Pregunta 10. ¿Costo del ensemble y cuándo se justifica?

Un ensemble de M modelos multiplica por M el costo de entrenamiento, memoria de almacenamiento e inferencia. Si cada modelo tiene P parámetros, el ensemble requiere $M \times P$ parámetros almacenados y M forward passes por predicción. Con $M=10$ y $P=2497$, el costo equivale a almacenar un modelo de 24.970 parámetros con la ventaja de que los errores son independientes entre modelos y se cancelan al promediar — si cada modelo comete errores con varianza σ^2 , el ensemble los reduce a σ^2/M . En este experimento, $n=10$ redujo el MAE de 1.370 a 1.292 (-5.7%) con rendimientos decrecientes ($n=3 \rightarrow 5$ ganó más que $5 \rightarrow 10$). Se justifica cuando: (1) el costo clínico del error es alto y mejoras del 5-10% tienen impacto real; (2) se necesita cuantificar incertidumbre mediante la varianza entre predicciones; (3) el tiempo de entrenamiento no es crítico. No se justifica cuando una regularización bien calibrada logra la misma mejora con un solo modelo.

Parte 7: Red neuronal con salidas mixtas

20. Motivación y construcción de targets

Se extiende el problema para que la red produzca simultáneamente tres predicciones de naturaleza distinta a partir de las mismas cinco variables clínicas, reflejando el contexto real donde el clínico necesita múltiples respuestas coherentes en una sola consulta.

Target	Naturaleza	Proceso generador	Dominio	Significado clínico
T1 — Cambio fisiológico	Continuo	$0.012 \cdot \text{edad} - 0.8 \cdot \text{dosis} + 0.015 \cdot \text{horas} + 0.4 \cdot \text{ir} + N(0,0.5^2)$	$\mathbb{R}$	Mejora (+) o deterioro (-) respecto a basal
T2 — Biomarcador	Positivo	$\exp(g(x) + \epsilon)$, mismo proceso Partes 1-5	$\mathbb{R}^+$	Concentración plasmática (ng/mL)
T3 — Estado clínico	Categorico	Umbral es sobre T1 y T2 (percentil 25/75)	$\{0,1,2\}$	Estable / Observación / Crítico

20.1 Reglas de asignación del estado clínico

- ▶ Crítico (2): biomarcador $\geq$ percentil 75 OR cambio < -1.5
- ▶ Observación (1): biomarcador $\in [p25, p75)$ OR cambio < 0
- ▶ Estable (0): biomarcador $< p25$ AND cambio ≥ 0

Las reglas se aplican en orden de prioridad descendente. La distribución resultante en train es aproximadamente: Estable 13%, Observación 55%, Crítico 32%.

21. Arquitectura: tronco compartido + tres cabezas

Módulo	Dimensiones	Parámetros	Activación
Tronco — Capa 1	5 → 64	5×64+64 = 384	ReLU
Tronco — Capa 2	64 → 32	64×32+32 = 2080	ReLU
Cabecal 1 — Capa	32 → 16 → 1	32×16+16+16×1+1 = 545	ReLU / Lineal
Cabecal 2 — Capa	32 → 16 → 1	545	ReLU / Lineal → exp(μ)
Cabecal 3 — Capa	32 → 16 → 3	32×16+16+16×3+3 = 579	ReLU / Logits
TOTAL	—	~4.133	—

El tronco compartido (5→64→32) aprende representaciones latentes útiles para las tres tareas simultáneamente. Cada cabezal (32→16→salida) especializa esa representación para su tipo de output.

Entrada (5) → Linear(5,64)+ReLU → Linear(64,32)+ReLU [tronco: 2.464 params]		
↓	↓	↓
Linear(32,16)+ReLU	Linear(32,16)+ReLU	Linear(32,16)+ReLU
Linear(16,1)→p∈ℝ	Linear(16,1)→exp(p)	Linear(16,3)→softmax

21.1 Garantía de positividad para T2

El cabezal 2 predice $\mu_i \in \mathbb{R}$ con salida lineal. La predicción final es $\hat{y}_i = \exp(\mu_i) > 0$ para todo μ_i , sin restricciones arquitectónicas adicionales. La pérdida NLL log-normal actúa sobre $(\log y_i - \mu_i)^2$.

22. Función de pérdida total

$$\begin{aligned} L_{\text{total}} &= \lambda_1 \cdot L_{\text{MSE}} + \lambda_2 \cdot L_{\text{NLL}} + \lambda_3 \cdot L_{\text{CE}} \\ &= 1.0 \cdot (1/I) \sum (\Delta_i - \hat{\Delta}_i)^2 + 5.0 \cdot (1/I) \sum (\log y_i - \mu_i)^2 + 1.0 \cdot \text{CE}(\text{logits}, \text{clase}) \end{aligned}$$

Salida	Pérdida	λ	Escala típica	Justificación
Cambio fisiológico	MSE	$\lambda_1=1.0$	0.3–1.0	Errores simétricos, dominio $\mathbb{R}$
Biomarcador	NLL log-normal	$\lambda_2=5.0$	0.05–0.15	Positiva, derivada en Parte 3
Estado clínico	Cross-entropy	$\lambda_3=1.0$	0.5–1.1	Multiclase, verosimilitud categórica

22.1 Calibración de los pesos λ

Se usa la estrategia de pérdida unitaria al inicio: λ calibrados para contribución similar en las primeras épocas. Dado que $L_{\text{MSE}} \sim 0.5$, $L_{\text{NLL}} \sim 0.10$ y $L_{\text{CE}} \sim 1.0$ al inicio, se elige $\lambda_2=5.0$ para que la NLL tenga peso comparable a las otras. Los valores se verificaron monitoreando que las tres pérdidas de validación descendieran simultáneamente en las primeras 50 épocas.

23. Resultados

Figura 7.1 — Métricas por salida en el conjunto de prueba.

— Salida 1: Cambio fisiológico —				
MAE:	0.4762			
RMSE:	0.5826			
Correlación:	0.8515			
— Salida 2: Biomarcador (positivo) —				
MAE:	1.4556			
Error Relativo Medio:	0.2600			
Predicciones negativas:	0			
— Salida 3: Clasificación clínica —				
Exactitud (accuracy):	0.7444			
Reporte por clase:				
	precision	recall	f1-score	support
Estable	0.80	0.70	0.74	23
Observación	0.75	0.80	0.77	99
Crítico	0.71	0.67	0.69	58
accuracy			0.74	180
macro avg	0.75	0.72	0.74	180
weighted avg	0.74	0.74	0.74	180

Figura 7.2 — Comparación numérica: red mixta vs modelos separados.

Comparación: Red Mixta vs Modelos Separados		
Métrica	Mixta	Separado
Cambio – MAE	0.4762	0.4920
Cambio – RMSE	0.5826	0.6119
Biomarcador – MAE	1.4556	1.6084
Biomarcador – E.Relativo	0.2600	0.2914
Clasificación – Accuracy	0.7444	0.7722
Parámetros totales	~3200	~7500

Tabla 7.1 — Métricas comparativas: red mixta vs modelos separados (n=180 test).

Salida	Métrica	Red Mixta	Modelo Separado	Δ
Cambio	MAE	0.4762	0.4920	+3.3% mejor
	RMSE	0.5826	0.6119	+4.8% mejor
	Correlación Pearson	0.8515	—	—
Biomarcador	MAE	1.4556	1.6084	+9.5% mejor
	E.Relativo	0.2600	0.2914	+10.8% mejor
	Preds negativas	0	0	—
Clasificación	Accuracy	0.7444	0.7722	-3.6% peor
	F1 macro	0.74	—	—
Eficiencia	Parámetros	~4.133	~7.500	-45% parámetros

Tabla 7.2 — Reporte por clase — Clasificación clínica.

Clase	n soporte	Precisión	Recall	F1
Estable (0)	23	0.80	0.70	0.74
Observación (1)	99	0.75	0.80	0.77
Crítico (2)	58	0.71	0.67	0.69
Macro avg	180	0.75	0.72	0.74

24. Respuestas a las preguntas del enunciado

24.1 Función de pérdida por salida

Salida	Pérdida	λ	Escala típica	Justificación
Cambio fisiológico	MSE	$\lambda_1=1.0$	0.3–1.0	Errores simétricos, dominio $\mathbb{R}$
Biomarcador	NLL log-normal	$\lambda_2=5.0$	0.05–0.15	Positiva, derivada en Parte 3
Estado clínico	Cross-entropy	$\lambda_3=1.0$	0.5–1.1	Multiclase, verosimilitud categórica

24.2 Garantía de positividad

El cabezal 2 predice μ_i con salida lineal irrestricta. La predicción en escala original es $\hat{y}_i=\exp(\mu_i)>0$ para todo $\mu_i\in\mathbb{R}$. Sin restricciones arquitectónicas (no se usa softplus, ReLU ni tope). Resultado: 0 predicciones negativas sobre el conjunto de prueba.

24.3 Calibración de λ

$\lambda_1=1.0$, $\lambda_2=5.0$, $\lambda_3=1.0$. El valor $\lambda_2=5.0$ compensa la escala $\sim 5\text{--}7\times$ menor de la NLL log-normal. Los λ se ajustaron monitoreando que las tres pérdidas de validación descendieran simultáneamente en las primeras épocas.

24.4 Métricas por salida

T1 (cambio): MAE=0.4762, RMSE=0.5826, correlación=0.8515. T2 (biomarcador): MAE=1.4556, MRE=0.2600, preds negativas=0. T3 (clasificación): accuracy=0.7444, F1 macro=0.74.

24.5 Conflicto entre tareas

Se observa conflicto parcial y asimétrico. Las tareas de regresión se benefician del tronco compartido: T1 mejora 4.8% en RMSE y T2 mejora 10.8% en error relativo respecto a modelos separados. La clasificación pierde 3.6% en accuracy (74.4% vs 77.2%). El tronco aprende representaciones optimizadas para predecir valores continuos, que no son las mejores para separar categorías. Mitigación: incrementar λ_3 o agregar una capa de representación exclusiva para el cabezal categórico.

24.6 Tronco compartido vs modelos separados

Aspecto	Tronco compartido	Modelos separados
Parámetros	~ 4.133	~ 7.500 (3 redes)
Transferencia	Confirmada: +10.8% biomarcador	No existe
Coherencia	Garantizada internamente	No garantizada
Conflicto grad.	Observado en clasificación	No existe
Calibración λ	Requerida	Sin hiperparámetros extra

La coherencia interna del tronco compartido es clínicamente relevante: un modelo único no puede predecir simultáneamente biomarcador bajo y estado crítico — las predicciones son internamente consistentes. Con tres modelos separados, esta coherencia no está garantizada.

Conclusiones y recomendaciones

25. Síntesis de hallazgos

25.1 Función de pérdida

La elección de la función de pérdida es la decisión de diseño con mayor impacto en este problema. El cambio de MSE a NLL log-normal — una modificación de una línea de código — redujo el error relativo en pacientes con concentraciones bajas ($y < 2$) de 54.4% a 28.3%, eliminó las 3 predicciones negativas y mejoró el MAE global en un 10.8%. Esto confirma que la coherencia entre la función de pérdida y el proceso generador de los datos tiene impacto clínico concreto.

25.2 Regularización

L2 con $\lambda = 5 \times 10^{-3}$ fue la estrategia individual más efectiva (MAE=1.173, -14.3%). L1 con $\lambda = 1 \times 10^{-3}$ fue competitiva (MAE=1.189). La regularización explícita superó sistemáticamente a la implícita (minibatch) y heurística (augmentation) para este problema. Data augmentation fue contraproducente en datos tabulares clínicos — una advertencia importante para la transferencia de técnicas de visión a dominios estructurados.

25.3 Ruido y sesgo-varianza

El análisis de ruido reveló que con $\sigma \geq 0.50$, el error irreducible ($\text{piso} = \sigma^2 = 0.25$) domina y la elección de arquitectura importa menos que la cantidad de datos y la regularización. La relación entre tamaño de dataset y varianza no es monótona para $n \leq 1200$, pero con $n = 2400$ la std entre semillas colapsa a 0.04-0.08 — evidencia de que más datos estabiliza el aprendizaje más que cualquier estrategia de regularización.

25.4 Red con salidas mixtas

El tronco compartido logró transferencia positiva para las tareas de regresión (+10.8% en biomarcador) con 45% menos parámetros que tres modelos separados, a costo de conflicto en la tarea de clasificación (-3.6%). La coherencia interna garantizada por el modelo único es una ventaja clínica que las métricas individuales no capturan.

26. Recomendaciones por restricción clínica

Tabla 8.1 — Modelo recomendado según el contexto de uso.

Restricción clínica	Modelo recomendado	MAE	Justificación
Máxima precisión, recurso disponible	Ensemble $n=10$ + L2	~1.17	Sin preds negativas, menor error
Un solo modelo, precisión alta	Log-normal + L2 $\lambda=5e-3$	1.173	Mejor individual, coherente
Incertidumbre por paciente	Log-normal σ^2 aprendido	1.480	Intervalos de predicción
Múltiples salidas simultáneas	Red mixta tronco compartido	1.456 bio	Coherencia interna garantizada
Despliegue en tiempo real	Log-normal $\sigma^2=1$ + L2 $\lambda=5e-3$	1.173	Un solo modelo, rápido
Datos ruidosos ($\sigma > 0.3$)	Cualquier + más datos	Piso= σ^2	Regularización ayuda, datos reducen varianza

26.1 Restricciones de implementación

- ▶ Para despliegue en tiempo real (latencia < 10 ms): Log-normal $\sigma^2=1 + L2 \lambda=5e-3$. Un solo forward pass, 2.497 parámetros.
- ▶ Para investigación clínica con necesidad de incertidumbre: Log-normal σ^2 aprendido. Requiere early stopping para estabilidad.
- ▶ Para sistema de soporte a decisiones multi-objetivo: Red mixta con tronco compartido. Aumentar λ_3 si la clasificación es prioritaria.
- ▶ Para entornos con muchos datos ($n>5000$): El ensemble pierde relevancia — un modelo único bien regularizado converge a resultados similares con menor costo.
- ▶ Para entornos con pocos datos ($n<300$): Ensemble + L2 fuerte. La varianza entre semillas es alta y el ensemble la compensa.

Declaración de uso de Inteligencia Artificial: > Durante la preparación y desarrollo de este trabajo, el autor utilizó el modelo de lenguaje Claude (Anthropic) como herramienta de asistencia técnica. Su uso se enfocó específicamente en la optimización del código fuente y en el refinamiento de ideas durante la etapa de diseño de la solución. El autor ha revisado, validado y asume la total responsabilidad por el contenido, la lógica final y los resultados presentados en este documento.